COSC 465: Computer Networks

Capstone Project: Final Report

Harun Mohamed, Jahanvi Chamria, Rahul Nair

ML-based Congestion Control with a Q/A Interface

Introduction

Modern home, school, and small-business networks simultaneously handle a mixture of traffic types. One laptop itself may be running a 4K video stream on Neltflix, a Zoom call, a large file download, and background software updates all at once. Traditional routers manage this either without QoS (Quality of Service) or by using static QoS rules that are coded in advance and do not adapt to changing conditions (e.g., different applications causing additional congestion/packet transfers). For example, if a user starts downloading a file that takes up a large portion of the bandwidth on a link, VoIP and video conferencing quality degrade, causing the user to experience jitter, dropped packets, and degraded call quality. Most users are unaware of why congestion causes this and have barely any means to understand why their specific network has these issues or why their call is lagging.

Our project addresses this gap by developing a network analysis system that uses machine learning (ML) to classify traffic types and detect congestion patterns from packet captures. The system can provide this information to the average user through a rule-based natural-language Q&A interface that we designed and implemented from scratch. A user can ask questions such as “Why is my video call lagging?” or “Which application is using the most bandwidth?” and receive simple answers with actionable recommendations based on the classifier outputs.

Our project involves evaluating the performance of multiple ML models on pre-recorded and labeled network captures from 5 traffic categories, with different window sizes, and train/test splits. This results in an analysis pipeline that makes network diagnostics accessible for non-technical users.

Objectives

We had three main technical objectives in this project:

1. Collect and process network packet captures using Wireshark, extracting flow-level features from recorded traffic across various application categories (VoIP, video streaming, file transfer, web browsing, and email), without requiring access to encrypted content.
2. Use machine learning (ML) to:
    a. Classify traffic flows into the above application categories
    b. Detect congestion states from short-time window aggregated packet captures, evaluated across different metrics (window sizes, train/test splits)
    c. Test the trained models on induced congestion captures (across three packet loss levels 5%, 10%, and 20%)
3. Provide a Q&A interface built from scratch (without external language models or API calls), that takes the output from both classifiers and uses rules and response templates to answer simple questions about the network state with actionable recommendations for the user.

These artifacts together form an end-to-end diagnostics pipeline:

Raw packet captures (live from the user) → labelled feature windows → ML classifiers → testing → natural-language explanation

It is important to note that, for the purposes of this report and the evaluation of our artifacts, we are not using live data captured from the user at this time. This is because the Raspberry Pi heats up and crashes if run for too long. We are testing our classifiers on testing data we captured, but the pipeline would also work equally well for any live data containing raw packet captures from a user.

System Design/Plan

A) Overall Plan

We have two ML pipelines that feed into a common Q&A interface:

- Traffic Classification Pipeline: This uses a pre-existing flow-feature dataset (ISCXVPN2016) [2][6] to train ML models, with the most accurate being XGBoost. The fine-tuned model can then identify application categories (VoIP, video streaming, file transfer, web browsing, and email) from network flow statistics such as IAT, packet counts, and byte counts.
- Congestion Detection Pipeline: This uses raw packet captures from Wireshark, cleans them, and aggregates them into specific time windows. Then it trains a binary classifier to distinguish congested from non-congested windows. Window size can be modified, and we evaluated 1-second, 3-second and 5-second windows. We also evaluated different train/test splits (20%, 30% used for testing)
- Packet Loss Impact on Detection: This uses a capture with different induced levels of packet loss (5%, 10% and 20%) to test if trained models can detect congestion even without concurrent apps running.
- Rule-Based Natural Language Q&A Interface: This receives the structured classifier outputs and uses certain thresholds and network conditions, along with keyword identifiers to determine what the user wants to know and then produces natural-language answers accordingly, without any external models or API calls.

B) Scripts (Total 10)

- ISCXVPN2016 data → format_classification_dataset.py → test_classifier.py
- TShark data → clean_raw_tshark_data.py → combine_clean_data.py → aggregate_label_packets.py → cong_classification_models.py → cv_test.py → test_model_by_traffic.py
- TShark data → clean_raw_tshark_data.py → label_congestion_amount.py → aggregate_label_packets.py → test_model_on_loss.py
- test_classifier.py & cong_classification_models.py data → qa_interface.py

C) Considerations

- We were only able to capture data for short durations (~15 minutes) as the Pi would heat up and crash. We decided to stop the data captures at around 700,000 packets every time to avoid this.
- When we introduced congestion, we observed changes in packet transmission. The number of packets captured was reduced to approximately half of the number recorded without any congestion.
- We decided to keep the traffic classifier and the congestion detection classifier separate. This is because the two work best with different models (XGBoost and Random Forest, respectively) and different parameters for those models. Forcing one model to evaluate both would reduce our overall accuracy.
- However, the outputs from both classifiers relate to each other, giving a broader picture of the network. For example, if the classifier returns 4 different types of traffic running simultaneously and congestion is also detected, we can relate the two outputs. We use the outputs from both to answer user questions in the Q&A interface, based on values of RTT, active flows, retransmissions etc.
- File names: Our data is on the shared Google Drive folder, and for our results the files are named with the first number showing the window size in seconds and the second number showing the testing split percentage. Eg. model_comparison_congestion_1_20 = 1-second windows, 30% test split.

Data Collection

Traffic Classification

The traffic classification model was trained on the ISCX nonVPN dataset published by CIC (https://www.unb.ca/cic/datasets/vpn.html) [2][6]. The file TimeBasedFeatures-Dataset-15s-NO-VPN.arff was used, which has time-based flow features aggregated over 15-second windows for seven application categories (BROWSING, CHAT, FILE TRANSFER, MAIL, P2P, STREAMING, and VOIP). This ARFF

file format was converted to CSV using format_classification_dataset.py, which loads the dataset as a pandas dataframe, decodes any byte strings to normal strings if needed, and then saves the data as a CSV file. The training-testing split was 80/20.

Congestion Detection

Network Congestion Simulation Commands

- To collect labeled training data, we used two Linux traffic control commands to programmatically switch the Raspberry Pi between healthy and congested network states.

Applying Congestion

- sudo tc qdisc add dev wlan0 root netem delay 250ms 50ms loss 5%

  This command artificially degraded outgoing traffic on the Pi's wireless interface by introducing a 250ms delay (±50ms jitter) and 5% random packet loss. We used this to create a reproducible "stress state," giving the model clear examples of the statistical signatures associated with network congestion.

Resetting to Baseline

- sudo tc qdisc del dev wlan0 root

  This removed all traffic shaping rules instantly, returning the Pi to normal operation. Without it, we could not have captured clean "recovery state" data following each congestion period.

Workflow Summary

- By sequencing these three commands, we implemented a standardized Healthy → Congested → Healthy testing cycle. This structured approach ensured our dataset contained clearly labeled examples of all three network states, making it suitable for training and evaluating the classification model.

To capture our own congestion data, we first performed a full capture without any congestion, with different types of traffic running simultaneously. Then, we did the same capture with congestion (delay=250ms and loss=5%). To train our model, we first split both datasets into 1-second windows and

labeled them as congestion/non-congestion. Next, we randomly aggregated the two datasets by ignoring the time labels and then tested our models on the aggregated data.

However, based on the feedback received for the beta artifacts, we decided it would be better to capture the data separately for each application type and to measure the specific window where we add congestion and the window where we remove it. We did 7 more data captures:

i) 5 for browsing, Gmail/chat, VoIP, 4k video streaming, and file downloads. For each one of these, we captured data with only that specific activity for 30 seconds, then added congestion for 1 minute, and then removed congestion for 30 seconds again. We decided to run all 5 application categories separately to reduce confounding from different applications running each time we captured. This makes our training process more standardized.

ii) Next, we captured the extent of congestion by doing 2 activities (browsing and 4k video streaming) at the same time and adding a higher level of congestion, then removing it, sequentially for every 30-second window. Our collection sequence was: no congestion → delay 250ms 5% loss → no congestion → delay 250ms 10% loss → no congestion → delay 250ms 20% loss.

iii) Lastly, we captured the effect of traffic mix on congestion by first opening all 5 activities one-by-one after 30-second delays, then adding congestion, and then closing the apps one-by-one after 30-second delays. This was our sequence: browsing → gmail/chat → youtube → VoIP → file download (100 mb) → ADD CONGESTION → stop file download → stop VoIP → stop youtube video → stop sending emails → stop browsing (each after 30 seconds).

We used this command to convert the TShark PCAP data to CSV in clean_raw_tshark_data.py:

 # & "C:\Program Files\Wireshark\tshark.exe" -r "File path" -T fields -e frame.time_epoch -e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport -e udp.srcport -e udp.dstport -e tcp.stream -e udp.stream -e frame.len -e tcp.len -e udp.length -e ip.proto -e tcp.flags -e tcp.window_size_value -e tcp.analysis.retransmission -e tcp.analysis.duplicate_ack -e tcp.analysis.lost_segment -e tcp.analysis.bytes_in_flight -e

tcp.analysis.ack_rtt -e dns.qry.name -e tls.handshake.extensions_server_name -E "header=y" -E "separator=," -E "quote=d" -E "occurrence=f" > "where you want your output\filename.csv"

In this file, we also replaced blanks with NaN, converted numeric columns, merged TCP/UDP port and stream columns, and created is_tcp/is_udp flags. We also assigned 0/1 to retransmissions/duplicate ACKs to make it easier to train our models.

These are the number of packets recorded for each capture:

- file_download = 217,404 packets
- gmail_chat = 101,440 packets
- streaming = 73,277 packets
- browsing = 72,096 packets
- voice_vid_call = 37,058 packets
- mix_capture = 318,992 packets (all 5 traffic types running simultaneously, time-range based labelling and congestion during specific times only).

Observations

While capturing the data, we also noticed the following:

- During the VoIP capture, we joined a Google Meet and enabled video for both users. The user with congestion had significant jitter and could not see/hear the other user properly, whereas the person without congestion could easily see and hear the other person. The Wireshark screen also showed many retransmissions. After removing the congestion, the call returned to good quality on both ends.
- When we added congestion to a 1080p YouTube video, the video started buffering a lot, and there were also a lot of retransmissions when we switched to another video during this period. When congestion was removed, this issue was fixed.

- When we tried to download a 100 MB file, the estimated time was around 2 minutes. However, it became 13 hours once we added congestion.
- The overall network speed was 3.5 Mbps across all our tests, indicating that the system experienced many packet losses and retransmissions.
- These observations were interesting, as we could correlate them with how and when we added or removed congestion.
- The number of packets captured with congestion added was roughly half (~230,647) compared to those captured without congestion (~589,620), even in the same 5 minute time period.

Labelling & Organization

We tried two modes: whole-capture labelling (all packets get the same label) and time-range labelling (sepcific intervals in seconds are labelled for when congestion was added, which allowed us to have a single capture with both scenarios). We did this for both congestion as well as traffic type labelling.

combine_clean_data.py merges all six cleaned CSVs into a single combined_dataset.csv with 820,267 packets. Each row has a source_file column that tells us which capture it came from.

aggregate_label_packets.py then groups packets based on time_windows (1-second, 3-second and 5-second) and then computes 23 summary features per window. The label for whether or not that window is congested is determined by the majority indicated by the packets in that window (if >50% were during a congestion interval, then the label was congested). Traffic type labels were assigned using “max”, so a window can have multiple traffic types too.

We had a capture for measuring congestion in different levels of packet loss: 5% loss (32-63 sec), 10% packet loss (92-122 sec), 20% loss (154-205 sec). Label_congestion_amount.py reads the cleaned data and adds a column packet_loss_level marking these levels (0, 0.05, 0.10, or 0.20).

Choice of Features

We had 4 categories with a total of 23 features per window:

Feature	Description	Category
total_packets	Number of packets in the 1-second window	Traffic volume
total_bytes	Total traffic volume in bytes	Traffic volume
mean_packet_size	Avg. packet size in bytes	Traffic volume
max_packet_size	Largest packet in the window	Traffic volume
std_packet_size	Standard deviation of packet sizes	Traffic volume
active_flows	Number of unique flows active (connections)	Flow behavior
retransmissions	Number of TCP retransmissions	Congestion signal
duplicate_acks	Number of duplicate ACKs	Congestion signal
lost_segments	Number of lost segments	Congestion signal
mean_rtt	Average RTT seen in window	Delay/latency
max_rtt	Maximum RTT observed in window	Delay/latency
std_rtt	Standard deviation of RTT values	Delay/latency
mean_bytes_in_flight	Average unacknowledged TCP bytes in flight	Network load/queue
max_bytes_in_flight	Peak unacknowledged TCP bytes in flight	Network load/queue
mean_window_size	Mean TCP receive window size	Network load/queue
tcp_packet_count	Number of TCP packets	Protocol
udp_packet_count	Number of UDP packets	Protocol
packets_per_sec	total_packets / window size	Derived feature

bytes_per_sec	total_bytes / window size	Derived feature
tcp_ratio	tcp_packet_count / total_apckets	Derived feature
udp_ratio	ud_packet_count / total_packets	Derived feature
avg_bytes_per_flow	total_bytes / active_flows	Derived per-flow
avg_packets_per_flow	total_packets / active_flows	Derived per-flow

Important: The source_file, time_window, window_index, and all 5 traffic type columns are excluded from model training so that the model learns about congestion from these features only, and not by knowing which window the data was in or which application was running.

- Prior work (Section 3.2) [1] identifies congestion using features such as packet delay, packet loss rate, and variations in packet timing. In our approach, we approximate these signals using packet-level statistics aggregated over time windows. For example, retransmissions and lost segments reflect packet loss, RTT captures delay, and bytes-in-flight indicates queue buildup.
- This prior work [1] also looks at congestion by analyzing data over multiple time steps, based on the assumption that network conditions don't change drastically over very short periods of time. Because of this, choosing the right time window matters. If the window is too small, there may not be enough data to actually capture meaningful congestion patterns. On the other hand, if the window is too large, it can smooth over short-term changes and hide important behavior. This is why we are testing a few windows (in a later section) to see which is most effective and not expanding too large since we don't have a lot of data.

ML Methodology

Traffic Classification

We compared four models: Decision Tree, KNN, SVM, and XGBoost. Each has a pipeline that fills missing values with the column median and uses StandardScale() for SVM and KNN because they are

sensitive to feature scaling. String labels were converted to numbers using the LabelEncoder(). XGBoost was selected as the final model after parameter fine-tuning with GridSearchCV (cv=5) and weighted F1 scoring. The optimal parameters found are:

- n_estimators=550
- max_depth=8
- learning_rate=0.19
- subsample=0.85
- colsample_bytree=0.92
- reg_lambda=1.0 (L2 regularization)

We also planned to retrain this fine-tuned XGBoost model on our flow feature data, and run it on a novel capture to see if it can identify the type of traffic. However, we had 2 issues with this:

- The dataset [2][6] had 15-second aggregates of time windows and different features from the features we extracted and labelled for congestion (described above in detail). It was difficult to figure out how to recalculate values for columns exclusively in the dataset [2][6] but ones that we did not use for congestion detection, and did not capture directly, such as IAT. The other option would be to retrain this model on our data but it would require another set of scripts for cleaning the data differently and fine-tune the parameters again for our captures.

Congestion Detection

We compared eight models on the aggregated congestion dataset: Logistic Regression, Decision Tree, Random Forest, Extra Trees, KNN, SVM, Gradient Boosting, and XGBoost. We had the same process, where each pipeline fills missing column values with the column median and uses StandardScale() for Logistic Regression, SVM and KNN because they are sensitive to feature scaling. Three window sizes (1-second, 3-second, and 5-second) and test split (20%, 30%) were evaluated. The best models were saved using joblib.

Cross-Validation

We ran 5-fold cross-validation (cv_test.py) on the full aggregated dataset using the saved best model. We evaluated this for different window sizes and testing splits. We also did this to check performance based on traffic type and whether a window contained a single traffic type or multiple types together.

Packet Loss Test

We used the saved models in test_model_on_loss.py to check how they perform on the pLoss dataset. It shows how detection accuracy changes depending on the amount of packet loss (5%, 10% or 20%) (which shows the congestion level). This also helps validate our model for unseen data.

Testing Results

Traffic classification (on the online dataset)

XGBoost had an overall test accuracy of 95.2% across all 7 traffic categories. VoIP and P2P had the highest precision and recall.

Category	Precision	Recall	F1 Score	Support
BROWSING	0.93	0.99	0.96	500
CHAT	0.94	0.88	0.91	178
FT	0.89	0.91	0.90	204
MAIL	0.88	0.84	0.86	50
P2P	0.98	0.97	0.98	200
STREAMING	0.93	0.79	0.85	96
VOIP	1.00	0.99	0.99	565

Overall Accuracy:

XGBoost (95.2%) > Decision Tree (90.5%) > KNN (86.3%) > SVM (71.6%)

This wide range of accuracies shows that gradient boosting for this kind of flow data is effective and consistent with existing literature.

Congestion detection

(Since we have multiple datasets and results, we chose to put the important takeaway results in the report, but all the data is in the shared Google Drive folder).

1) Congestion with 1-Second Windows (20% Test Split)

Model	Accuracy	Precision	Recall	F1
Random Forest	0.9004739336	0.8878504673	0.9134615385	0.9004739336
XGBoost	0.8957345972	0.8727272727	0.9230769231	0.8971962617
Gradient Boosting	0.8672985782	0.8454545455	0.8942307692	0.8691588785
Extra Trees	0.8436018957	0.8659793814	0.8076923077	0.8358208955
Decision Tree	0.8199052133	0.8113207547	0.8269230769	0.819047619
KNN	0.7725118483	0.7857142857	0.7403846154	0.7623762376
SVM	0.7677725118	0.816091954	0.6826923077	0.7434554974
Logistic Regression	0.7393364929	0.7247706422	0.7596153846	0.7417840376

Random forest confusion matrix: 95 true negatives, 12 false positives, 9 false negatives, 95 true positives.

The low accuracy of logistic regression shows that the congestion was non-linear.

2) Congestion with 5-Second Windows (30% Test Split) (Best Result)

Model	Accuracy	Precision	Recall	F1
Random Forest	0.9545454545	0.9393939394	0.96875	0.9538461538
Gradient Boosting	0.9393939394	0.9117647059	0.96875	0.9393939394

XGBoost	0.9393939394	0.9117647059	0.96875	0.9393939394
Extra Trees	0.9242424242	0.8857142857	0.96875	0.9253731343
Decision Tree	0.8636363636	0.8484848485	0.875	0.8615384615
Logistic Regression	0.7424242424	0.6530612245	1	0.7901234568
KNN	0.7424242424	0.6744186047	0.90625	0.7733333333
SVM	0.6818181818	0.6037735849	1	0.7529411765

Here, Random Forest had better accuracy than the 1-second windows.

3) Window Size Comparison (Random Forest only)

For 30% testing:

Window	Accuracy	Precision	Recall	F1
1 second	0.9179810726	0.9171974522	0.9171974522	0.9171974522
3 seconds	0.9082568807	0.9038461538	0.9038461538	0.9038461538
5 seconds	0.9545454545	0.9393939394	0.96875	0.9538461538

For 20% testing:

Window	Accuracy	Precision	Recall	F1
1 second	0.9004739336	0.8878504673	0.9134615385	0.9004739336
3 seconds	0.9178082192	0.8918918919	0.9428571429	0.9166666667
5 seconds	0.9545454545	0.9523809524	0.9523809524	0.9523809524

These values do not show much of a consistent trend, however in general, 5-second windows perform better. This might be due to the fact that slightly longer windows allow more RTTs to be seen, more lost packets to be seen, and more retransmissions data compared to 1-second windows. This data also shows that the model performs better with 30% testing data compared to 20% testing data.

4) Cross-Validation (5s, 30% Best Random Forest Model)

Fold	Accuracy	Precision	Recall	F1
1	0.8636363636	0.8947368421	0.8095238095	0.85
2	0.9090909091	0.8695652174	0.9523809524	0.9090909091
3	0.9772727273	0.9545454545	1	0.976744186
4	0.8863636364	0.9444444444	0.8095238095	0.8717948718
5	0.8181818182	0.9375	0.6818181818	0.7894736842
Mean	0.8909090909	0.9201583917	0.8506493506	0.8794207302
Std	0.05882808684	0.03632815127	0.1270251147	0.06958156857

The mean F1 value is 87.9% with a standard deviation of 6.9%. This might mean that the performance of the model varies depending on the capture in every fold. For example, maybe split 3 had only congested windows.

5) Performance By Traffic Type (5s, 30% Best Random Forest Model)

Traffic Type	Samples	Accuracy	Precision	Recall	F1
traffic_email_chat	85	0.9529411765	0.9318181818	0.9761904762	0.9534883721
traffic_browsing	115	0.9391304348	0.9482758621	0.9322033898	0.9401709402
traffic_streaming	69	0.9275362319	0.9696969697	0.8888888889	0.9275362319
traffic_voice_video_call	58	0.8965517241	0.9310344828	0.8709677419	0.9
traffic_file_transfer	47	0.8936170213	0.9130434783	0.875	0.8936170213

All have F1 above 89%. Email/chat windows are classified the most accurately, while file transfer and VoIP are the most difficult to classify accurately.

6) Single and Mixed Traffic Windows (5s, 30% Best Random Forest Model)

Group	Samples	Accuracy	Precision	Recall	F1
Single Traffic Type	161	0.8571428571	0.8955223881	0.7894736842	0.8391608392
Mixed Traffic Types	59	0.9830508475	0.9677419355	1	0.9836065574

Mixed traffic windows (with multiple applications active simultaneously) are easier to classify correctly (accuracy=98.3%), which makes sense because there are more packets and more congestion consistently when many apps run together than single traffic types (accuracy=85.7%), where congestion might increase and decrease over time.

7) Packet Loss (Impact of Level of Congestion)

In label_congestion_amount.py, the capture with different levels of congestion was cleaned, labelled, and aggregated into 5-second windows.

Model	Samples	Accuracy	Precision	Recall	F1
5s, 20% split	41	0.8536585366	0.8695652174	0.8695652174	0.8695652174
5s, 30% split	41	0.8536585366	0.84	0.9130434783	0.875

By loss level (5s, 30% only):

Packet Loss Level	Samples	Accuracy	Precision	Recall	F1	Observations
0%	17	0.7647058824	0	0	0	3 False positives!
5%	7	1	1	1	1	Good detection
10%	7	1	1	1	1	Good detection
20%	10	0.8	1	0.8	0.8888888889	Even though highest loss, some windows undetected

At 5 and 10%, the model has good classification (moderate losses have enough retransmissions and ACKs for detection). However, it does not perform as well for 20% loss, maybe because the sender might be sending less packets, reducing in-flight bytes. At 0% loss, the model has 3 false positives, which shows that the model is somewhat over-sensitive when used on a new, unseen capture.

Q&A Interface

For the Q&A interface, we used a rule-based system using conditional logic and keyword matching along with pre-written response templates. It does not use any external model or API calls. The interface takes

the context of a network, then applies the conditions, and fills the responses depending on the user's question. The interface uses keyword like lag/slow, call, congestion, bandwidth, and health summary etc. to determine what the user wants to know. NOTE: We focused more on the ML congestion part of the project, so we did not test out the interface. Some examples of how we wanted it to work:

User: *Why is my video call lagging?*

System: *"Congestion was detected in this capture (model confidence: 95%).*

A significant portion of windows (42%) had bulk traffic (Streaming or File Transfer) active. This type of traffic competes with latency-sensitive applications like video calls for bandwidth. Retransmission rate: 3.2% – packets are being dropped and resent, which directly causes jitter in VoIP calls. Recommendation: Pause or reschedule large downloads and streaming sessions during calls to reduce competition on the link."

User: *Give me a health summary.*

System: *"=== Network State Summary === Status: CONGESTED (confidence: 95%). Dominant traffic: Streaming. Avg throughput: 4.2 MB/s. Avg RTT: 87 ms. Retransmit rate: 3.2%. Bulk traffic active in 42% of windows. Latency-sensitive traffic active in 17% of windows."*

Limitations/Sources of Error

- Congestion Dataset Size: We trained and tested on relatively small time windows (ranging from a few minutes to around 10 minutes). This was due to the number of packets the Pi could handle at once (~700,000). This limited our ability to train the models on generalized, mixed congestion over a longer period. Longer captures with more congestion/non-congestion windows and activities would possibly improve our accuracy.
- Training and Testing on Same Data: Although we tested with various training/testing splits for our data, in general we have been using the same kind of data (that we captured sequentially) to

both train and test so our accuracy might be slightly inflated compared to if we ran the models on data from another device with actual online activity that a user might do.

- Not Running on Real-Time Data: We are currently capturing data separately and then running our models on it, and not using live user data because our setup to do that is on the Pi and needs to be controlled and exported every time; it cannot be kept running for long periods.
- Traffic Prioritization Only Based on Recommendations: We currently analyze the network state and make recommendations, but do not actually prioritize the apps we want or modify packet scheduling. This would require a different approach at the OS level.
- Traffic Single-Capture Point: We are capturing data from only one network interface on a single device. The system can also have multiple devices connected to the Pi's hotspot at the same time, which would change how packets arrive.
- Human Error in Labeling: When capturing all our data, we kept a stopwatch to log when we added/removed congestion (for labeling). Those time laps have a ±2-second error because three people were doing different things while capturing data.
- Q&A Interface: The interface can only handle a finite set of predefined question categories. Questions that fall outside those categories or ones that are phrased in a way that is not included in the keyword matching part may not be answered. A generative model would be able to handle this better, but it might introduce hallucinations and would also require all the data before giving an answer. For our rule-based interface, we could expand the accepted keyword list and add more response templates to improve our answers.

Future Improvements

- Running the traffic classifier or our captured data (by retraining the model on our features)
- Live Data: To use the system live, we would connect both trained models to a live packet-capture script to assess real-time network traffic. The aggregate_label_packets.py script could continuously update values in order to update classifier outputs.

- Interface: We could add more keywords and responses to the interface to cover additional edge cases, such as specific applications or alerts based on certain thresholds.
- Simulation of Prioritization: We could have a software simulation of actual prioritization, even if we do not actually implement it on the router.

References

[1] Chengze Du, Zhiwei Yu, and Xiangyu Wang. 2024. Identification of Path Congestion Status for Network Performance Tomography using Deep Spatial-Temporal Learning. (December 2024). DOI:https://doi.org/10.48550/arXiv.2412.10762

[2] Gerard Draper-Gil, Arash Habibi Lashkari, Mohammad Saiful Islam Mamun, and Ali A. Ghorbani. 2016. Characterization of Encrypted and VPN Traffic using Time-related Features. *Proceedings of the 2nd International Conference on Information Systems Security and Privacy* (2016). DOI:https://doi.org/10.5220/0005740704070414

[3] Ramazan Enisoglu and Veselin Rakocevic. 2023. Low-Latency Internet Traffic Identification using Machine Learning with Trend-based Features. *2023 International Wireless Communications and Mobile Computing (IWCMC)* (June 2023), 394–399. DOI:https://doi.org/10.1109/iwcmc58020.2023.10183084

[4] Rehab H. Serag, Mohamed S. Abdalzaher, Hussein Abd El Atty Elsayed, and M. Sobh. 2025. Software Defined Network Traffic Classification for QoS Optimization Using Machine Learning. *Journal of Network and Systems Management* 33, 2 (February 2025). DOI:https://doi.org/10.1007/s10922-025-09911-6

[5] Pu Wang, Shih-Chun Lin, and Min Luo. 2016. A Framework for QoS-aware Traffic Classification Using Semi-supervised Machine Learning in SDNs. (June 2016). DOI:https://doi.org/10.1109/scc.2016.133

[6] VPN 2016 | Datasets | Research | Canadian Institute for Cybersecurity | UNB. *www.unb.ca*. Retrieved June 17, 2023 from https://www.unb.ca/cic/datasets/vpn.html